Optimización de Rutas de Distribución Multi-Depósito mediante Recocido Simulado (Simulated Annealing) para Cadenas de Restaurantes en Culiacán, Sinaloa

Cervantes Araujo Carlos Iván

Manjarrez Olivas Mariana

Carrera: Ingeniería en Sistemas Computacionales

Docente: M.C. Zuriel Datan Mora

Instituto Tecnológico de Culiacán

Asignatura: Tópicos de Inteligencia Artificial

22 de octubre de 2026

Resumen

El Recocido Simulado redujo la distancia total de reparto de una cadena de restaurantes en Culiacán de **421.78 km a 205.35 km (51.31 % menos)**, con la misma reducción en tiempo estimado y costo de combustible.

La red tiene 10 centros de distribución y 90 sucursales. Cada sucursal se asignó a su centro más cercano y, para cada centro, se resolvió un Problema del Viajante (TSP) con Recocido Simulado programado desde cero en Python (Google Colab). La ruta de referencia visita las sucursales en el orden de la tabla. Las pruebas con cinco semillas aleatorias dieron mejoras de entre 49.82 % y 51.76 %, lo que confirma que el método es estable aunque no garantiza el óptimo global.

Introducción

Planear el orden de visita de las sucursales es la decisión que más pesa en los kilómetros que recorre cada vehículo de reparto. Una cadena de restaurantes en Culiacán, Sinaloa, surte diariamente abarrotes (arroz, frijol, aceite, enlatados) desde sus centros de distribución a sus sucursales: cada vehículo sale de un centro, visita varias sucursales y regresa al mismo centro.

Con pocas sucursales se pueden probar todas las combinaciones. Con 20 sucursales ya existen más de 10^{18} rutas posibles, por lo que revisarlas todas no es práctico. Por eso se usa una metaheurística: un método que encuentra soluciones muy buenas en poco tiempo sin explorar todo el espacio.

Este trabajo aplica el Recocido Simulado (Simulated Annealing) a la red logística urbana de la cadena y compara la ruta optimizada contra una ruta de referencia en distancia, tiempo y costo de combustible.

Planteamiento del problema

El problema es encontrar, para cada uno de los 10 centros de distribución, el orden de visita de sus sucursales que minimice la distancia total recorrida. Se trata de un problema de ruteo multi-depósito con 100 puntos en total.

Datos utilizados

Documento	Archivo	Contenido

Documento	Archivo	Contenido
1. Tabla de distribución	datos_distribucion_tiendas.xlsx	100 puntos: 10 centros de distribución y 90 sucursales, con nombre, coordenadas, capacidades y nivel (A, B, C)
2. Matriz de distancias	matriz_distancias.xlsx	Distancia entre cada par de puntos (100 × 100), en km
3. Matriz de costos de combustible	matriz_costos_combustible.xlsx	Costo de combustible entre cada par de puntos (100 × 100), en MXN

La fila *i* de la tabla corresponde a la columna Nodo_i de las dos matrices.

Observaciones sobre los datos

- Ambas matrices son simétricas (ir de A a B cuesta lo mismo que de B a A) y tienen ceros en la diagonal.
- En todos los pares, el costo de combustible es 0.15 MXN por km. Por lo tanto, la ruta más corta también es la más barata.
- Las distancias de la matriz no coinciden con las coordenadas (latitud/longitud) de la tabla: hay puntos casi juntos en el mapa que en la matriz aparecen a varios kilómetros. Se tomó la matriz de distancias como fuente oficial y no se dibujaron las rutas sobre un mapa.
- La tabla no indica la demanda de cada sucursal ni la capacidad de los vehículos, así que no se modelan restricciones de carga.

Objetivos

Objetivo general: implementar el algoritmo de Recocido Simulado para encontrar rutas de distribución más cortas entre los centros de distribución y las sucursales de la cadena.

Objetivos específicos:

1. Cargar y analizar los datos de sucursales, distancias y costos de combustible.
2. Modelar el problema como un Problema del Viajante (TSP) por centro de distribución.
3. Programar el Recocido Simulado desde cero en Python.

4. Comparar una ruta inicial de referencia contra la ruta optimizada en distancia, tiempo y costo.

Marco teórico

Problema del Viajante (TSP) y ruteo multi-depósito

El TSP busca el recorrido más corto que visita cada punto exactamente una vez y regresa al origen. Es un problema NP-difícil: con *n* sucursales hay (n − 1)!/2 recorridos distintos, así que la búsqueda exhaustiva deja de ser viable con pocas decenas de puntos. Cuando hay varios depósitos, el problema se conoce como ruteo de vehículos multi-depósito (MDVRP); una forma común de abordarlo es agrupar los clientes por depósito y resolver un TSP en cada grupo.

Recocido Simulado

El nombre viene del recocido de metales: el metal se calienta y se enfría poco a poco para que sus átomos se acomoden en un estado de menor energía. En el algoritmo, la "energía" es la distancia de la ruta. Una ruta mejor siempre se acepta; una ruta peor se acepta con la probabilidad de Metrópolis:

```
P = e^{-\Delta E / T}
```

Donde ΔE es cuánto empeora la distancia y T la temperatura actual. Con T = 1000, una ruta 5 km peor se acepta con P ≈ 0.995 (el algoritmo explora); con T = 1, P ≈ 0.007 (el algoritmo afina). La temperatura baja geométricamente, T = T × α, hasta un mínimo.

Comparación con Búsqueda Tabú

	Recocido Simulado	Búsqueda Tabú
Idea	Acepta a veces soluciones peores según una probabilidad que baja con la temperatura	Se mueve al mejor vecino y prohíbe regresar a soluciones recientes (lista tabú)
Ventajas	Fácil de programar; evalúa un solo vecino por paso; escapa de óptimos locales	Búsqueda más dirigida; suele converger rápido
Desventajas	Es aleatorio; hay que ajustar temperatura y enfriamiento	Revisa muchos vecinos por iteración; más código

Se eligió Recocido Simulado porque es sencillo de implementar, explora combinaciones de forma aleatoria, acepta temporalmente soluciones peores para escapar de óptimos locales, no necesita evaluar todas las rutas y cada paso se entiende fácilmente, lo que lo hace apropiado para una implementación educativa.

Modelo matemático y metodología

Cada centro se resuelve como un TSP independiente que minimiza la distancia de su ruta cerrada. Para un centro *c* con sucursales asignadas s_1, …, s_n, una ruta es:

$$\text{ruta} = [\,c,\; s_{(1)},\; s_{(2)},\; \dots,\; s_{(n)},\; c\,]$$

Función objetivo (minimizar):

$$D(\text{ruta}) = \sum_{k=1}^{n+1} d\big(\text{ruta}_{k-1},\, \text{ruta}_{k}\big)$$

- c: centro de distribución, inicio y fin de la ruta.
- $s_{(k)}$: sucursal visitada en la posición k.
- n: número de sucursales que atiende el centro.
- d(i, j): distancia en km entre los puntos i y j, tomada de la matriz de distancias.

Restricciones: cada sucursal se visita exactamente una vez, y la ruta empieza y termina en su centro de distribución. El costo de combustible y el tiempo de entrega no se optimizan directamente; se calculan después como métricas de evaluación.

Pasos de la metodología

1. **Asignación:** cada sucursal se asigna al centro con menor distancia en la matriz. Así ningún centro se ignora y no se mezclan sucursales entre centros.
2. **Ruta de referencia:** cada vehículo visita sus sucursales en el orden en que aparecen en la tabla (Tienda 1, Tienda 2, …). Es válida, pero nadie la planeó.
3. **Optimización:** se aplica Recocido Simulado a la ruta de referencia de cada centro.
4. **Evaluación:** se comparan distancia, tiempo y costo, y se validan las rutas con pruebas automáticas.

Asignación resultante

Centro de distribución	Sucursales asignadas
CD 4	21

Centro de distribución	Sucursales asignadas
CD 7	17
CD 9	12
CD 5	11
CD 6	8
CD 2	6
CD 1	5
CD 3	4
CD 10	4
CD 8	2
Total	**90**

Supuesto de tiempo

Los documentos no incluyen velocidades. Se supuso una velocidad promedio urbana de 30 km/h en Culiacán; el tiempo solo considera traslado, sin descarga ni tráfico:

```
\text{tiempo (min)} = \frac{\text{distancia (km)}}{30} \times 60
```

Implementación del algoritmo

El algoritmo se programó desde cero en Python en Google Colab, usando solo random, math, pandas y matplotlib. Prueba unas 183,800 rutas vecinas por centro y tarda menos de un minuto en total.

Parámetros

Parámetro	Valor	Significado
Temperatura inicial	1000	Al inicio se aceptan casi todas las rutas peores
Temperatura mínima	0.1	Cuando la temperatura baja de aquí, el algoritmo termina

Parámetro	Valor	Significado
Factor de enfriamiento (α)	0.995	La temperatura baja 0.5 % en cada paso
Intentos por temperatura	100	Rutas vecinas probadas antes de enfriar
Semilla aleatoria	42	Permite repetir los resultados

Con estos valores la temperatura baja en 1,838 pasos (de 1000 a 0.1), que funcionan como límite de iteraciones. Con un solo vecino por paso, el algoritmo haría muy pocos intentos para un grupo de 20 sucursales; por eso se prueban 100 por temperatura.

Generación de vecinos

Se eligen dos posiciones al azar entre 1 y len(ruta) − 2 y se intercambian las sucursales. Así el centro de distribución (primera y última posición) nunca se mueve. Ejemplo: [0, 11, 12, 13, 14, 0] → [0, 11, 13, 12, 14, 0].

```
def generar_vecino(ruta):
    vecina = ruta.copy()
    if len(ruta) < 4:
        return vecina
    i, j = random.sample(range(1, len(ruta) - 1), 2)
    vecina[i], vecina[j] = vecina[j], vecina[i]
    return vecina
```

Función de distancia

Suma cada tramo de la ruta usando la matriz de distancias; como la ruta incluye el centro al final, el regreso al origen se suma automáticamente. La función de costo es idéntica pero consulta la matriz de costos.

```
def calcular_distancia(ruta):
    distancia_total = 0
    for k in range(len(ruta) - 1):
        distancia_total = distancia_total + matriz_distancias[ruta[k]][ruta[k + 1]]
    return distancia_total
```

Recocido Simulado

La función guarda la mejor ruta por separado para no perderla y registra un historial de la mejor distancia en cada paso de temperatura.

```
def recocido_simulado(ruta_inicial):
    ruta_actual = ruta_inicial.copy()
    distancia_actual = calcular_distancia(ruta_actual)
    mejor_ruta = ruta_actual.copy()
    mejor_distancia = distancia_actual
    temperatura = TEMPERATURA_INICIAL
    historial = []

    while temperatura > TEMPERATURA_MINIMA:
        for intento in range(INTENTOS_POR_TEMPERATURA):
            ruta_vecina = generar_vecino(ruta_actual)
            distancia_vecina = calcular_distancia(ruta_vecina)
            delta = distancia_vecina - distancia_actual

            if delta < 0:
                aceptar = True
            else:
                probabilidad = math.exp(-delta / temperatura)
                aceptar = random.random() < probabilidad

            if aceptar:
                ruta_actual = ruta_vecina
                distancia_actual = distancia_vecina
                if distancia_actual < mejor_distancia:
                    mejor_ruta = ruta_actual.copy()
                    mejor_distancia = distancia_actual

        historial.append(mejor_distancia)
        temperatura = temperatura * FACTOR_ENFRIAMIENTO

    return mejor_ruta, mejor_distancia, historial
```

Pruebas de validación

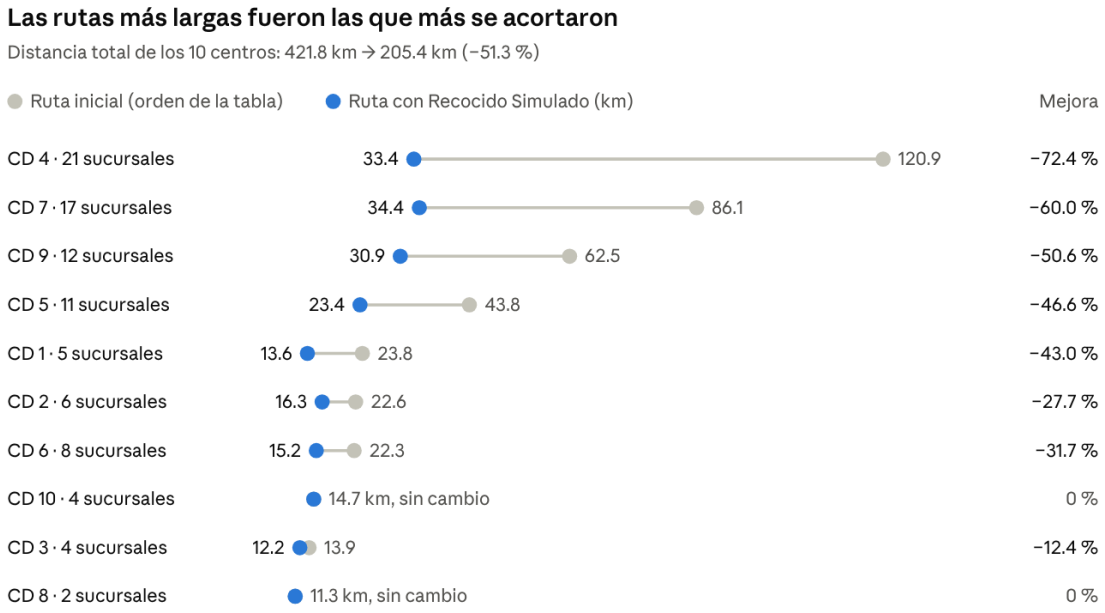
Todas las pruebas con assert se cumplieron:

- Cada ruta inicia y termina en su centro de distribución.
- Cada ruta visita exactamente sus sucursales asignadas, sin repetir.

- Ninguna ruta optimizada es peor que la inicial.
- Las 90 sucursales se visitan exactamente una vez en total.
- La función de distancia incluye el regreso al origen.
- La distancia y el costo se leen de su matriz correcta.
- La mejor distancia nunca empeora en el historial.

Resultados y análisis

Con la semilla 42, el Recocido Simulado redujo la distancia total de 421.78 km a 205.35 km, un ahorro de 216.43 km (51.31 %).



Proyecto2_Recocido_Simulado.ipynb · semilla 42 · distancias de matriz_distancias.xlsx

La mejora crece con el tamaño del grupo: el CD 4, con 21 sucursales, pasó de 120.9 a 33.4 km. Los CD 8 y CD 10 no mejoraron porque su ruta inicial ya era la óptima: con 2 sucursales el orden no cambia la distancia (la matriz es simétrica), y con 4 solo hay 3 recorridos distintos.

Distancia, tiempo y costo

Métrica	Ruta inicial	Ruta optimizada	Mejora
Distancia total (km)	421.78	205.35	51.31 %

Métrica	Ruta inicial	Ruta optimizada	Mejora
Tiempo estimado (min, a 30 km/h)	843.57	410.70	51.31 %
Costo de combustible (MXN)	63.27	30.80	51.31 %

La mejora es igual en las tres métricas porque el tiempo (velocidad constante) y el costo (0.15 MXN/km) son proporcionales a la distancia.

Ejemplo: centro con más sucursales (CD 4, 21 sucursales)

- **Ruta inicial (120.90 km):** CD 4 → T3 → T7 → T21 → T32 → T35 → T37 → T39 → T43 → T44 → T46 → T51 → T52 → T57 → T59 → T63 → T75 → T78 → T80 → T82 → T83 → T84 → CD 4
- **Mejor ruta encontrada (33.41 km):** CD 4 → T35 → T84 → T39 → T52 → T21 → T46 → T63 → T44 → T80 → T7 → T43 → T75 → T51 → T78 → T3 → T83 → T82 → T57 → T59 → T32 → T37 → CD 4

Rutas optimizadas de los 10 centros

Centro	Ruta optimizada
CD 1	CD 1 → T38 → T34 → T49 → T30 → T45 → CD 1
CD 2	CD 2 → T1 → T6 → T74 → T81 → T47 → T69 → CD 2
CD 3	CD 3 → T17 → T88 → T79 → T26 → CD 3
CD 4	Ver el ejemplo anterior
CD 5	CD 5 → T66 → T11 → T65 → T70 → T55 → T50 → T13 → T53 → T41 → T86 → T71 → CD 5
CD 6	CD 6 → T87 → T15 → T62 → T76 → T9 → T33 → T10 → T16 → CD 6
CD 7	CD 7 → T89 → T27 → T2 → T90 → T5 → T67 → T48 → T31 → T60 → T14 → T54 → T77 → T85 → T56 → T22 → T25 → T73 → CD 7
CD 8	CD 8 → T64 → T24 → CD 8
CD 9	CD 9 → T8 → T68 → T42 → T18 → T20 → T12 → T40 → T36 → T19 → T29 → T72 → T58 → CD 9

Centro	Ruta optimizada
CD 10	CD 10 → T4 → T23 → T28 → T61 → CD 10

Estabilidad con diferentes semillas

Semilla	Distancia optimizada total (km)	Mejora
2024	203.47	51.76 %
42	205.35	51.31 %
100	205.35	51.31 %
1	205.42	51.30 %
7	211.64	49.82 %

Todas las semillas mejoran alrededor de 50 %, así que el algoritmo es estable. La semilla 2024 encontró una ruta 1.88 km más corta que la semilla 42: el Recocido Simulado da soluciones muy buenas en poco tiempo, pero no garantiza el óptimo global.

Conclusiones y trabajo futuro

El Recocido Simulado cumplió el objetivo: redujo a menos de la mitad los kilómetros, el tiempo y el costo de combustible de la red de 10 centros y 90 sucursales.

1. El problema de distribución se modeló como un TSP por centro de distribución, tras asignar cada sucursal a su centro más cercano.
2. El Recocido Simulado redujo la distancia total de 421.78 km a 205.35 km (51.31 %) frente a la ruta de referencia.
3. Como el costo (0.15 MXN/km) y el tiempo (30 km/h) dependen de la distancia, reducir kilómetros redujo en la misma proporción el costo y el tiempo de entrega.
4. Aceptar rutas peores al inicio (temperatura alta) evitó quedarse en soluciones mediocres; al final (temperatura baja) el algoritmo solo aceptó mejoras.
5. Con cinco semillas distintas las mejoras fueron de 49.82 % a 51.76 %: el método es estable, pero no garantiza el óptimo global.

- vecindarios como 2-opt y comparar contra Búsqueda Tabú con los mismos datos.

Referencias

1. Kirkpatrick, S., Gelatt, C. D. y Vecchi, M. P. (1983). Optimization by Simulated Annealing. *Science*, 220(4598), 671–680.
2. Metropolis, N., Rosenbluth, A. W., Rosenbluth, M. N., Teller, A. H. y Teller, E. (1953). Equation of State Calculations by Fast Computing Machines. *The Journal of Chemical Physics*, 21(6), 1087–1092.
3. Laporte, G. (1992). The vehicle routing problem: An overview of exact and approximate algorithms. *European Journal of Operational Research*, 59(3), 345–358.
4. Glover, F. (1986). Future paths for integer programming and links to artificial intelligence. *Computers & Operations Research*, 13(5), 533–549.
5. Datos del proyecto: datos_distribucion_tiendas.xlsx, matriz_distancias.xlsx y matriz_costos_combustible.xlsx; implementación en Proyecto2_Recocido_Simulado.ipynb (Google Colab).